Week 1 — Python Basics for Data Analysis

Industrial Training TR-103 · Data Analytics Organisation: World With Web **Trainee:** Savreet Kaur · URN 2302668 **Period:** 15 June 2026 – 19 June 2026

Objective of the week

To build the Python foundation that every later topic depends on — the language basics, then the two libraries that do the actual analytical work, NumPy and pandas. By the end of the week a raw CSV file can be loaded, inspected and described without leaving the notebook.

Day 1 — Monday, 15 June 2026

Orientation and environment setup

The day began with an introduction to the training schedule, the expected deliverables, and the overall path the course would follow: Python first, then statistics, then SQL, and finally visualisation and Power BI.

The development environment was set up and verified:

- **Anaconda Distribution** installed, which bundles Python with the scientific libraries rather than requiring each to be installed separately.
- **Jupyter Notebook** launched and tested. Its cell-by-cell execution suits analysis work, where a dataset is loaded once and then explored through many small steps.
- **VS Code** configured with the Python extension as an editor for longer scripts.

Python fundamentals

TOPIC	COVERED
Variables	Dynamic typing, naming conventions
Numeric types	`int`, `float`, integer vs float division
Strings	Indexing, slicing, f-strings, common methods
Booleans	Comparison and logical operators
Type conversion	`int()`, `float()`, `str()`, and where it fails

```
name = "Savreet"
marks = 87.5
print(f"{name} scored {marks:.1f} marks")
```

Takeaway

Python's dynamic typing is convenient but means a column read from a CSV can silently be text when numbers were expected. Checking types early was emphasised as a habit, and it became relevant almost immediately in Week 2.

Day 2 — Tuesday, 16 June 2026

Control flow

Conditional statements (`if` / `elif` / `else`) and the two loop forms were covered, along with `break`, `continue` and the `range()` function.

```
for score in [45, 72, 91]:
    if score >= 90:
        grade = "A"
    elif score >= 60:
        grade = "B"
    else:
        grade = "C"
    print(score, grade)
```

Functions

- Defining functions with `def`, parameters and return values
- Default arguments and keyword arguments
- Scope: local versus global variables
- Docstrings, and why a function that is not documented is not reusable

A small exercise required writing a function that takes a list of numbers and returns the mean, ignoring any `None` values — a simplified version of the missing-data handling that Week 2 covers properly.

Takeaway

Writing analysis steps as small named functions makes a notebook readable. The alternative — one long cell — works until something has to be changed.

Day 3 — Wednesday, 17 June 2026

Data structures

STRUCTURE	ORDERED	MUTABLE	TYPICAL USE IN ANALYSIS
List	Yes	Yes	A column of values before conversion
Tuple	Yes	No	Fixed records, function returns
Dictionary	Yes (3.7+)	Yes	Column name → values; JSON-like data
Set	No	Yes	Unique values, membership tests

List comprehensions

A compact way to transform and filter a sequence in one expression:

```
scores = [45, 72, 91, 58]
passed = [s for s in scores if s >= 60]
scaled = [s / 100 for s in scores]
```

Practice

A small dataset of student records was represented as a list of dictionaries, then queried with comprehensions — filtering, extracting a single field, and counting by category. This is deliberately the same shape of problem that pandas solves later, so the value of pandas is visible by comparison.

Takeaway

Comprehensions are readable up to about one condition. Beyond that a normal loop — or, later, a pandas operation — is clearer.

Day 4 — Thursday, 18 June 2026

NumPy

NumPy introduces the `ndarray`, a fixed-type array that supports vectorised operations.

```
import numpy as np

a = np.array([45, 72, 91, 58])
print(a.mean(), a.std(), a.max())
print(a * 1.1)          # scales every element, no loop
print(a[a >= 60])       # boolean masking
```

Topics covered:

- Creating arrays: `np.array`, `np.zeros`, `np.arange`, `np.linspace`
- Array attributes: `shape`, `dtype`, `ndim`
- Vectorised arithmetic and broadcasting
- Boolean masking and `np.where`
- Aggregations: `sum`, `mean`, `std`, `min`, `max`, and the `axis` argument

Why vectorisation matters

A loop over a Python list executes the interpreter once per element. A NumPy operation executes compiled code once over the whole block of memory. Timing the two over 100,000 elements showed the difference clearly, which explains why pandas — built on NumPy — is fast on large tables.

Takeaway

Boolean masking is the single most useful idea of the day: it is the same mechanism that filters rows in pandas and the `WHERE` clause in SQL.

Day 5 — Friday, 19 June 2026

pandas: Series and DataFrame

- **Series** — a one-dimensional labelled array, essentially one column.
- **DataFrame** — a two-dimensional labelled table, a dictionary of Series.

Loading data

```
import pandas as pd

df = pd.read_csv("sales.csv")
xl = pd.read_excel("sales.xlsx", sheet_name="Q1")
```

Useful parameters covered: `sep`, `header`, `names`, `usecols`, `nrows`, `parse_dates` and `dtype`.

First inspection of a dataset

A routine was established for opening any new dataset, and it was used unchanged for the rest of the training:

STEP	COMMAND	QUESTION ANSWERED
1	`df.shape`	How many rows and columns?
2	`df.head()`	What does a row look like?
3	`df.info()`	What type is each column, and how many nulls?
4	`df.describe()`	What is the range of the numeric columns?
5	`df.isnull().sum()`	Where exactly is data missing?

Selecting data

- Column selection: `df["col"]`, `df[["a", "b"]]`
- Label-based rows: `df.loc[]`
- Position-based rows: `df.iloc[]`
- Conditional filtering: `df[df["sales"] > 1000]`

Takeaway

`df.info()` is the most informative single command in the whole workflow. It answers the type question and the missing-data question at once, which are the first two things that go wrong in any real dataset.

Summary of Week 1

DAY	DATE	TOPIC
1	Mon 15 Jun	Orientation, setup, Python basics
2	Tue 16 Jun	Control flow, loops, functions
3	Wed 17 Jun	Lists, dicts, tuples, sets, comprehensions
4	Thu 18 Jun	NumPy arrays, vectorisation, masking
5	Fri 19 Jun	pandas Series/DataFrame, loading and inspection

Skills gained: Python syntax and functions · NumPy vectorised operations · loading CSV and Excel data into pandas · inspecting an unfamiliar dataset.

Next week: cleaning the problems that inspection reveals, then describing the data statistically.